

Databricks Certified Generative AI Associate

100-Question Practice Set

Blueprint-weighted • MCQ + open Q&A • with answer explanations

Section (weight)	Qs
1 · Design Applications (14%)	14
2 · Data Preparation (14%)	14
3 · Application Development (30%)	30
4 · Assembling & Deploying Apps (22%)	22
5 · Governance, Security & Privacy (8%)	8
6 · Evaluation & Monitoring (12%)	12
TOTAL	100

Exam: 45 scored MCQs · 90 minutes · \$200 · 6 weighted sections

Companion practice set to your Domain 1–6 study guides

How To Use This Set

These 100 questions are distributed to mirror the official 2026 exam weighting, so the number of questions per section reflects how heavily each area is tested. Application Development (30%) and Assembling & Deploying (22%) carry the most weight — prioritise them.

Each item is either a **multiple-choice question (MCQ)** with options A–D and a marked correct answer, or an **open question** that prompts you to explain a concept in your own words before reading the model answer. Every item includes a short explanation covering *why the answer is right and why the distractors are wrong*.

Newer 2026-tested topics are woven throughout: **Agent Bricks, MCP (Model Context Protocol), AI Gateway (inference/usage tables, rate limiting), MLflow 3 Tracing, Prompt Registry/versioning, Mosaic AI Agent Evaluation, custom scorers, and Lakehouse Monitoring.**

Suggested approach: cover the answer, attempt the question, then check. Re-test any section where you score below ~80%. Ask for 100 more once you've worked through these.

Section 1 — Design Applications (14%)

Q1. MCQ

A team must decompose a complex 'analyze quarterly earnings calls and produce a risk summary' requirement into a GenAI solution. Which decomposition BEST reflects sound multi-stage design?

- A. One giant prompt that does transcription, extraction, and summarization at once
- B. Transcribe → extract key claims (extraction task) → retrieve related filings (RAG) → summarize risks (generation)
- C. Fine-tune a single decoder model end-to-end on all earnings calls
- D. Use an encoder-only model to generate the final risk narrative

✓ **Answer: B**

Problem decomposition breaks a complex requirement into discrete, individually-optimizable stages, each mapped to the right tool/model. A is brittle and unmaintainable; C is costly and inflexible; D is wrong because encoder-only models do not generate text.

Q2. MCQ

You need a prompt that reliably returns JSON with fields {sentiment, confidence}. Which element of the four-part prompt structure is MOST responsible for this?

- A. Instruction
- B. Context
- C. Input
- D. Output Format

✓ **Answer: D**

The Output Format component specifies structure, schema, and style. Adding an explicit JSON schema (and optionally a few-shot example) is the primary lever for consistent structured output. Instruction states the task but not the shape.

Q3. OPEN

What is the difference between zero-shot CoT and few-shot prompting, and when would you pick few-shot over zero-shot CoT?

🔗 **Model answer:**

Zero-shot CoT appends a reasoning trigger ('think step by step') with no examples; it helps multi-step reasoning. Few-shot supplies labeled examples to teach a pattern/format. Pick few-shot when the task needs a specific output FORMAT or domain pattern the model can't infer from instructions alone; pick zero-shot CoT when the challenge is reasoning rather than format.

Q4. MCQ

A requirement: classify 50M support tickets into 5 categories at high throughput and low cost, no text generation needed. Best architecture?

- A. Decoder-only LLM (Llama 3 70B) with few-shot prompts
- B. Encoder-only model (BERT-style) fine-tuned for classification
- C. Seq2Seq model (FLAN-T5)
- D. Multi-agent supervisor pattern

✓ **Answer: B**

Encoder-only models give bidirectional understanding optimized for classification at high throughput/low cost. A decoder LLM is slower and pricier for pure classification; Seq2Seq is overkill; multi-agent adds unneeded orchestration.

Q5. MCQ

Which scenario is the STRONGEST signal to choose a multi-agent (supervisor + workers) pattern over a single ReAct agent?

- A. A single tool needs to be called in a loop until success
- B. Independent subtasks (research, calculation, drafting) can run in parallel and be combined
- C. The model must output strictly formatted JSON
- D. You need to reduce token usage

✓ **Answer: B**

Parallel, specialized subtasks coordinated and merged is the canonical multi-agent signal. A describes a single ReAct loop; C is an output-format concern; D is unrelated to agent topology.

Q6. OPEN

In the ReAct pattern, what happens when a tool call returns an error, and why is this called 'self-correcting'?

🔗 **Model answer:**

The error is fed back as an Observation; the agent then generates a new Thought to decide whether to retry with different params, switch tools, or reformulate. The Thought→Action→Observation loop lets the agent reason about failures rather than crash — the self-correcting property.

Q7. MCQ

You must build a system that reads unstructured PDF contracts and returns structured JSON (parties, dates, amounts) at scale on Databricks. Which Agent Brick fits BEST?

- A. Knowledge Assistant Brick
- B. Multi-Agent Supervisor Brick
- C. Information Extraction Brick
- D. Genie Space

✓ **Answer: C**

The Information Extraction Brick is purpose-built to turn unstructured text into structured fields at scale. Knowledge Assistant is for Q&A; Supervisor coordinates agents; Genie is for natural-language analytics over tables.

Q8. MCQ

A prompt template will be reused across an app with user-provided input. What is the PRIMARY design risk to mitigate?

- A. Token cost
- B. Prompt injection via untrusted user input
- C. Choosing the wrong embedding model
- D. Latency

✓ **Answer: B**

Reusable templates that interpolate untrusted user input are vulnerable to prompt injection. Mitigations: input delimiting, instruction/role separation, validation, and guardrails. Cost and latency matter but are not the primary security risk of templating.

Q9. OPEN

Why are decoder-only models described as 'autoregressive,' and what does that imply about their suitability for generation vs classification?

🔗 **Model answer:**

Autoregressive = they generate one token at a time, each conditioned on previously generated tokens, using causal (left-to-right) attention. This makes them excellent at open-ended generation but, compared to bidirectional encoders, less naturally suited to whole-sequence classification where full bidirectional context helps.

Q10. MCQ

When designing a LangChain pipeline, which ordering of components is correct for a basic RAG chain?

- A. LLM → Retriever → Prompt Template → Output Parser
- B. Prompt Template → LLM → Retriever → Output Parser
- C. Retriever → Prompt Template → LLM → Output Parser
- D. Output Parser → Retriever → LLM → Prompt Template

✓ **Answer: C**

Retrieve relevant context first, inject it into the prompt template, send to the LLM, then parse the structured output. Retrieval must precede prompt assembly so the context is available to the template.

Q11. MCQ

A chatbot is hallucinating facts using only zero-shot prompting. Per increasing complexity, what is the NEXT step (not the final one)?

- A. Fine-tune the base model
- B. Add grounding context / system prompt with factual constraints
- C. Switch to a multi-agent architecture
- D. Implement RLHF

✓ **Answer: B**

Order of increasing complexity: add grounding context → RAG → few-shot with fact-checking → CoT with 'say I don't know.' The NEXT (cheapest) step is grounding context, not jumping to fine-tuning or RLHF.

Q12. OPEN

What distinguishes the Planning agentic pattern from ReAct?

✍ **Model answer:**

Planning first generates an explicit multi-step plan, then executes steps (often sequentially), optionally re-planning. ReAct interleaves reasoning and acting one step at a time without a full upfront plan. Planning suits decomposable sequential tasks; ReAct suits exploratory tool-driven loops.

Q13. MCQ

Which is the PRIMARY advantage of Agent Bricks over hand-built custom agents?

- A. Maximum flexibility to customize every step
- B. Faster time-to-production with built-in MLflow tracing and Vector Search integration
- C. Lower model inference cost
- D. They eliminate the need for evaluation

✓ **Answer: B**

Agent Bricks are production-ready, Databricks-maintained templates with built-in observability and integrations — the main benefit is speed-to-production and built-in tracing. Flexibility is the advantage of custom agents, not Bricks; they do not remove the need for evaluation.

Q14. MCQ

You need translation across 12 languages with full input read before output. Which architecture is the classic fit?

- A. Encoder-only (BERT)
- B. Decoder-only (GPT-style)
- C. Encoder-Decoder / Seq2Seq (mT5, FLAN-T5)
- D. Diffusion model

✓ **Answer: C**

Seq2Seq encodes the full input then decodes the full output — the canonical design for translation/summarization, especially multilingual (mT5). Modern large decoders can also translate, but the textbook/exam answer for read-fully-then-generate is encoder-decoder.

Section 2 — Data Preparation (14%)

Q15. MCQ

Your RAG answers miss context that spans across chunk boundaries. What is the simplest effective fix?

- A. Decrease chunk size to 50 tokens
- B. Add chunk overlap (e.g., 10–20%)
- C. Remove all metadata
- D. Switch to keyword search

✓ **Answer: B**

Overlap carries context across boundaries so a concept split between chunks is still retrievable. Shrinking chunks worsens fragmentation; removing metadata hurts filtering; keyword search abandons semantic retrieval.

Q16. OPEN

Explain the core trade-off in choosing chunk size for RAG.

✍ **Model answer:**

Large chunks preserve context but dilute embedding focus and may exceed the generator's context window or retrieve irrelevant material; small chunks are precise but lose surrounding context and can fragment ideas. The goal is the smallest chunk that still contains a self-contained idea, tuned with overlap.

Q17. MCQ

A corpus contains code, prose, and tables. Which chunking strategy generally preserves meaning BEST?

- A. Fixed character-count chunking ignoring structure
- B. Structure-aware / recursive chunking respecting natural boundaries (headings, functions, rows)
- C. One chunk per document regardless of length
- D. Random chunking

✓ **Answer: B**

Recursive/structure-aware splitting respects semantic boundaries (paragraphs, code blocks, table rows), keeping units coherent. Fixed-size cuts can split mid-sentence/mid-function; whole-doc chunks overflow context; random destroys meaning.

Q18. MCQ

You must extract clean text (and handle embedded images) from many PDFs in a RAG ingestion job with the LEAST code. Which Python package is most appropriate?

- A. beautifulsoup
- B. unstructured
- C. requests
- D. pillow

✓ **Answer: B**

`unstructured` parses many document types (incl. PDFs with mixed text/images) into clean elements with minimal code, ideal for RAG ingestion. BeautifulSoup is for HTML; requests is HTTP; Pillow is image processing only.

Q19. OPEN

Why does the embedding model used at ingestion time have to match the one used at query time?

🔗 **Model answer:**

Similarity search compares vectors in a shared geometric space. Embeddings from different models live in different spaces, so distances are meaningless across them. Both indexing and querying must use the same embedding model (and version) or retrieval quality collapses.

Q20. MCQ

Documents include a 'department' field used to restrict which chunks a user can retrieve. Where should this live?

- A. Concatenated into the chunk text only
- B. As metadata on each indexed record, used as a filter at query time
- C. In the embedding model weights
- D. Discarded to save space

✓ **Answer: B**

Storing 'department' as metadata enables metadata filtering (and access control) during retrieval. Burying it in text makes it unreliable to filter on; it cannot live in model weights; discarding it loses governance capability.

Q21. MCQ

When selecting an embedding model, which factor is MOST important for long legal documents?

- A. The model's maximum context (sequence) length
- B. Its training framework
- C. Its license color
- D. The number of GPUs it was trained on

✓ **Answer: A**

If chunks are long, the embedding model's max sequence length must accommodate them or text gets truncated and information is lost. Context length is the key selection driver for long-document chunks.

Q22. OPEN

What is the role of a Delta table in a Databricks RAG ingestion pipeline, and why is it preferred over raw files?

🔗 **Model answer:**

Delta tables provide ACID transactions, schema enforcement, versioning/time-travel, and Change Data Feed. In RAG, source/cleaned chunks land in Delta so updates are tracked and a Vector Search index can sync incrementally from the table. Raw files lack governance, versioning, and reliable incremental sync.

Q23. MCQ

Which pre-processing step most directly improves retrieval precision for a noisy web-scraped corpus?

- A. Increasing embedding dimensionality
- B. Cleaning boilerplate (nav bars, ads, repeated footers) before chunking
- C. Lowering the LLM temperature
- D. Adding more retrieved chunks (higher k)

✓ **Answer: B**

Removing boilerplate prevents junk from being embedded and retrieved, directly improving precision. Embedding size is model-fixed; temperature affects generation not retrieval; higher k often lowers precision.

Q24. MCQ

A team needs to keep their vector index fresh as source documents change daily. Best Databricks approach?

- A. Manually re-embed everything nightly via notebook
- B. Use a Delta Sync Vector Search index that auto-syncs from the source Delta table
- C. Store embeddings in a CSV and reload
- D. Disable the index and use full-text search

✓ **Answer: B**

A Delta Sync index keeps the vector index continuously/incrementally aligned with the source Delta table as it changes. Manual full re-embeds are costly and error-prone; CSV reloads lose governance; abandoning vectors loses semantic search.

Q25. OPEN

What problem does metadata filtering solve that pure semantic similarity does not?

🔗 **Model answer:**

Semantic similarity finds conceptually close chunks but can't enforce hard constraints (date ranges, document type, access tier, tenant). Metadata filtering applies exact, attribute-based restrictions — essential for correctness, recency, and access control — typically combined with vector search (hybrid).

Q26. MCQ

For a FAQ bot where each answer is short and self-contained, which chunking is usually best?

- A. Very large 4,000-token chunks
- B. One Q&A pair per chunk
- C. Character-split every 20 chars
- D. Whole knowledge base as one chunk

✓ **Answer: B**

FAQ entries are naturally atomic; one Q&A per chunk maximizes retrieval precision and keeps context self-contained. Oversized or whole-KB chunks dilute relevance; 20-char splits destroy meaning.

Q27. MCQ

You want hybrid search combining semantic and keyword matching on Databricks Vector Search. What does the keyword component primarily add?

- A. Better handling of exact terms, IDs, and rare tokens the embedding may blur
- B. Lower embedding cost
- C. Automatic chunking
- D. Model fine-tuning

✓ **Answer: A**

Hybrid search adds lexical/keyword matching that catches exact strings, product codes, and rare tokens where dense embeddings can be fuzzy, improving recall on precise terms. It doesn't change cost, chunking, or training.

Q28. OPEN

Why might you store both the raw chunk text and a cleaned/normalized version in your pipeline?

🔗 **Model answer:**

You embed/normalize one version for retrieval consistency but often want the original text to feed the generator for faithful, readable context and citations. Keeping both supports debugging, re-embedding with a new model, and traceability back to source.

Section 3 — Application Development (30%)

Q29. MCQ

You want to log a LangChain RAG chain so it can be served and tracked. Which MLflow approach is standard on Databricks?

- A. Pickle the chain and upload to DBFS
- B. Log it with MLflow (LangChain/PyFunc flavor) and register to Unity Catalog
- C. Save as a .txt prompt file
- D. Email the notebook to ops

✓ **Answer: B**

MLflow logs chains via the LangChain flavor (or a custom PyFunc), then you register the model to Unity Catalog and serve it. Pickling loses lineage/signatures and breaks serving; the other options aren't deployable models.

Q30. OPEN

What is an MLflow PyFunc model and why is it useful for custom GenAI applications?

🔗 **Model answer:**

PyFunc is a generic MLflow flavor wrapping arbitrary Python logic (predict method) with a standard interface, signature, and dependencies. It lets you package custom GenAI logic — pre/post-processing, multiple model calls, tool use — into one deployable, servable artifact when no built-in flavor fits.

Q31. MCQ

A code-assistant must support many programming languages with quality as the top priority. Among Databricks Foundation Model API options, which fits BEST?

- A. BGE-large
- B. Llama2-7B
- C. A strong code-specialized model (e.g., CodeLlama-class)
- D. DistilBERT

✓ **Answer: C**

A code-specialized model is purpose-built for multi-language code generation quality. BGE-large is an embedding model; DistilBERT is encoder-only (no generation); a small general 7B underperforms a code-tuned model on this task.

Q32. MCQ

Which Databricks feature lets you call hosted LLMs on a simple pay-per-token basis without managing endpoints?

- A. Provisioned Throughput endpoints
- B. Foundation Model APIs (pay-per-token)
- C. Unity Catalog Volumes
- D. Delta Live Tables

✓ **Answer: B**

Foundation Model APIs offer pay-per-token access to hosted models with no endpoint management — ideal for variable/low volume. Provisioned Throughput is for steady high volume with reserved capacity; Volumes and DLT are unrelated to LLM serving.

Q33. OPEN

When should you choose Provisioned Throughput over pay-per-token Foundation Model APIs?

🔗 **Model answer:**

Choose Provisioned Throughput when you have steady, high request volume and need predictable latency/throughput and cost, or guaranteed capacity/SLA. Pay-per-token suits spiky, low, or unpredictable volume where you don't want reserved capacity. If volume is too low to justify a provisioned endpoint, use pay-per-token.

Q34. MCQ

A RAG app's generator was swapped to a self-hosted model with a SHORTER context window and now errors on long prompts. Without changing the model, which TWO fixes apply? (choose the pair)

- A. Reduce number of retrieved chunks (lower k) AND decrease chunk size
- B. Use a smaller embedding model AND retrain with ALiBi
- C. Increase max output tokens AND add more retrieved chunks
- D. Switch to keyword-only search AND remove the prompt template

✓ **Answer: A**

The prompt is too long for the new context window. Retrieving fewer chunks and using smaller chunks both shrink the assembled prompt. Embedding-model size doesn't change prompt length; ALiBi requires retraining; raising output tokens or adding chunks makes it worse.

Q35. MCQ

You need an LLM to call an external function (e.g., `get_weather`) reliably. What capability are you using?

- A. Tool/function calling
- B. Quantization
- C. Tokenization
- D. Distillation

✓ **Answer: A**

Function/tool calling lets the model emit a structured call (name + arguments) that your app executes, returning results back to the model. Quantization, tokenization, and distillation are unrelated model-efficiency/representation concepts.

Q36. OPEN

What is MLflow's Prompt Registry and what exam-relevant problem does it solve?

🔗 **Model answer:**

The Prompt Registry stores prompts as versioned, named artifacts with lifecycle/aliases, enabling prompt versioning, comparison, rollback, and reuse across apps. It solves the problem of untracked prompt drift — letting teams manage prompts like code/models. The 2026 exam explicitly tests prompt versioning/lifecycle.

Q37. MCQ

A business chatbot must route each query to the right specialist model (billing vs technical vs sales). Which design implements this?

- A. A single static prompt
- B. A router/classifier step that dispatches to the appropriate model or chain
- C. Increasing temperature
- D. Disabling retrieval

✓ **Answer: B**

A routing layer classifies intent and forwards to the specialized model/chain — a common multi-stage pattern. A static prompt can't specialize; temperature and disabling retrieval don't address routing.

Q38. MCQ

In LangChain, what does an Output Parser contribute to a chain?

- A. Fetches documents from a vector store
- B. Converts the LLM's raw text into a structured object (e.g., JSON/Pydantic) and can enforce schema
- C. Encrypts the prompt
- D. Schedules the job

✓ **Answer: B**

Output Parsers transform raw model text into structured, validated objects and can trigger re-asks on parse failure. Retrieval is the Retriever's job; parsers don't encrypt or schedule.

Q39. OPEN

What is Mosaic AI Agent Framework and how does it relate to deploying tool-using agents?

🔗 **Model answer:**

The Mosaic AI Agent Framework is Databricks' toolkit for building, logging (via MLflow), evaluating, and deploying multi-step, tool-calling agents as serving endpoints with tracing and governance. It standardizes authoring agents and wiring them to Vector Search, Foundation Model APIs, and Agent Evaluation.

Q40. MCQ

Which parameter most directly controls randomness/creativity of an LLM's output?

- A. top_k retrieval
- B. temperature
- C. chunk overlap
- D. embedding dimension

✓ **Answer: B**

Temperature scales the sharpness of the token distribution: lower = more deterministic, higher = more random/creative. top_k retrieval is a RAG setting; overlap and embedding dim are data/representation parameters.

Q41. MCQ

You want short-term conversational memory in a chatbot so it remembers the last few turns. Which approach matches?

- A. Windowed/buffer memory injecting recent turns into the prompt
- B. Fine-tuning after every message
- C. Increasing the embedding model size
- D. Disabling the system prompt

✓ **Answer: A**

Windowed (buffer) memory keeps the most recent N turns and injects them as context — the standard short-term memory pattern. Fine-tuning per message is infeasible; embedding size and system prompts don't provide turn memory.

Q42. OPEN

Describe context injection as a design pattern and name two variants.

🔗 **Model answer:**

Context injection means inserting relevant information into the prompt at runtime. Variants: (1) full-history injection — include the entire conversation; (2) windowed injection — include only recent turns; (3) fact-store/knowledge-triple memory — retrieve only salient facts. The choice trades completeness against token cost and relevance.

Q43. MCQ

Which is the correct way to give an LLM access to proprietary, frequently-updated company data WITHOUT retraining?

- A. Fine-tuning weekly
- B. RAG over a synced vector index of that data
- C. Hard-coding facts in the system prompt forever
- D. Lowering temperature

✓ **Answer: B**

RAG retrieves current data at query time from a synced index, so updates appear without retraining. Frequent fine-tuning is costly/slow; static system-prompt facts go stale; temperature is irrelevant to knowledge freshness.

Q44. MCQ

What is the purpose of MLflow model signatures for a GenAI endpoint?

- A. They encrypt the model weights
- B. They define expected input/output schema, improving validation and serving reliability
- C. They set the GPU type
- D. They generate embeddings

✓ **Answer: B**

Signatures capture the input/output schema so serving can validate requests and tools can introspect the contract. They don't encrypt weights, choose hardware, or create embeddings.

Q45. OPEN

What is MCP (Model Context Protocol) and why is it relevant to building Databricks agents in 2026?

🔗 **Model answer:**

MCP is an open standard for connecting LLMs/agents to external tools and data sources through a uniform interface, so a model can discover and call tools/resources consistently. It's newly relevant because Databricks supports MCP for agent tool integration, letting agents access governed data and services in a standardized, interoperable way.

Q46. MCQ

A team adds guardrails to block toxic or off-topic responses in a finance chatbot. Which is NOT a valid guardrail technique?

- A. Content filters / safety classifiers on inputs and outputs
- B. System-prompt constraints and refusal instructions
- C. Topic/scope restriction via a domain classifier
- D. Increasing temperature to add variety

✓ **Answer: D**

Raising temperature increases randomness and would WORSEN safety/relevance — it is not a guardrail. Content filters, constrained system prompts, and topic classifiers are all valid guardrail techniques.

Q47. MCQ

You must summarize claims from many documents and want reusable prompt logic across notebooks. Best practice?

- A. Copy-paste the prompt string everywhere
- B. Use a parameterized prompt template (and register it in a prompt registry)
- C. Embed the prompt in the model weights
- D. Store the prompt in a screenshot

✓ **Answer: B**

Parameterized templates (plus a prompt registry) make prompts reusable, versioned, and maintainable. Copy-paste causes drift; prompts can't live in weights; screenshots aren't usable programmatically.

Q48. OPEN

Explain why 'tool ordering' matters in a sequential-logic agent and give an example.

🔗 **Model answer:**

When one tool's output feeds another, the agent must call them in the correct dependency order. Example: an insurance-claim agent must first call a 'lookup_policy' tool to get coverage limits before calling 'calculate_payout', because the payout depends on the policy data. Wrong ordering yields incorrect or impossible computations.

Q49. MCQ

Which statement about few-shot example count is correct for the exam?

- A. More examples always strictly improve accuracy with no downside
- B. More examples can improve pattern-following but increase tokens (cost + latency) and risk exceeding context
- C. Examples reduce token count
- D. Examples are only for classification

✓ **Answer: B**

Adding examples helps the model learn the pattern but raises token usage (cost, latency) and can hit context limits, with diminishing returns. Examples don't reduce tokens and aren't limited to classification.

Q50. MCQ

A RAG response is grammatical but contradicts the retrieved context. Which development fix targets this directly?

- A. Increase chunk overlap
- B. Add an instruction to answer ONLY from provided context and cite it, plus a faithfulness check
- C. Use a bigger embedding model
- D. Raise temperature

✓ **Answer: B**

Constraining the generator to the supplied context (and validating faithfulness) directly reduces context-contradiction. Overlap/embeddings affect retrieval, not generator faithfulness; higher temperature worsens it.

Q51. OPEN

What is the difference between a Retriever returning k=3 vs k=10 chunks, and how does it interact with the generator's context window?

🔗 **Model answer:**

Higher k increases recall (more chance the answer is present) but adds tokens and noise, can lower precision, and may overflow the generator's context window. Lower k is leaner and more precise but risks missing relevant context. Tune k against both retrieval quality and the model's context limit.

Q52. MCQ

Which Databricks construct serves a registered model as a REST endpoint with autoscaling?

- A. Unity Catalog Volume
- B. Model Serving endpoint
- C. Delta Live Table
- D. Photon

✓ **Answer: B**

Model Serving exposes a registered model as a scalable REST endpoint. Volumes store files; DLT builds pipelines; Photon is a query engine — none serve models.

Q53. MCQ

To reduce hallucination in tool-using agents, why is structured tool output (typed schema) helpful?

- A. It removes the need for any LLM
- B. It gives the model precise, parseable grounding rather than free-text it might misread
- C. It increases temperature
- D. It disables retrieval

✓ **Answer: B**

Typed, structured tool results are unambiguous and easy to ground on, reducing misinterpretation and hallucination. It doesn't eliminate the LLM, change temperature, or disable retrieval.

Q54. OPEN

What is the role of an embedding endpoint vs a chat/completions endpoint in a Databricks RAG app?

🔗 **Model answer:**

The embedding endpoint converts text (queries and documents) into vectors for similarity search; the chat/completions endpoint generates the final natural-language answer from the retrieved context. They are separate Foundation Model API endpoints serving distinct stages of the pipeline.

Q55. MCQ

A developer wants deterministic outputs for a regression-style test of their prompt. Which setting helps MOST?

- A. temperature = 0 (and fixed seed where supported)
- B. temperature = 1.5
- C. top_p = 1.0 with temperature 2
- D. Random sampling enabled

✓ **Answer: A**

Temperature 0 (greedy decoding) maximizes determinism, aiding reproducible tests. Higher temperatures and broad sampling increase variability, undermining test stability.

Q56. MCQ

Which is the BEST description of a 'chain' in LangChain?

- A. A blockchain ledger of prompts
- B. A composed sequence of components (prompts, models, parsers, retrievers) where each step's output feeds the next
- C. A GPU cluster
- D. A type of embedding

✓ **Answer: B**

A chain composes modular steps into a pipeline, passing outputs forward. It is not a blockchain, hardware, or an embedding type.

Q57. OPEN

Give two reasons to wrap a multi-component GenAI app as a single MLflow model before serving.

🔗 **Model answer:**

(1) Atomic deployment & versioning — the whole app (retrieval, prompt, model calls, parsing) is logged, registered, and rolled out as one governed artifact with a signature and dependencies. (2) Consistent serving & observability — one endpoint with unified input/output contract, MLflow tracing, and Unity Catalog lineage, instead of brittle hand-wired services.

Q58. MCQ

An agent must decide at runtime whether to use a calculator tool or answer directly. Which pattern is this?

- A. Tool Use pattern (model chooses if/when to invoke a tool)
- B. Fixed pipeline
- C. Pure classification
- D. Batch inference

✓ **Answer: A**

The Tool Use pattern lets the model decide whether and which tool to call based on the query. A fixed pipeline removes that choice; classification and batch inference don't describe dynamic tool selection.

Section 4 — Assembling & Deploying Applications (22%)

Q59. MCQ

What is the EASIEST correct process to deploy an LLM already trained on Databricks?

- A. Pickle it, upload to a Volume, register via MLflow, start an endpoint
- B. Log it with MLflow during training, register directly to Unity Catalog via the MLflow API, start a serving endpoint
- C. Build a Docker image and run a container manually
- D. Wrap it in Flask and serve with Gunicorn on a VM

✓ **Answer: B**

The native path is: MLflow-log → register to Unity Catalog → Model Serving endpoint, preserving lineage, signatures, and governance with least friction. Pickling, manual Docker, and Flask/Gunicorn bypass the managed, governed workflow.

Q60. OPEN

Walk through the canonical end-to-end RAG pipeline order on Databricks and name the responsible component at each stage.

✍ **Model answer:**

1) Ingest/clean source docs → Delta tables. 2) Chunk the text. 3) Embed chunks via an embedding endpoint. 4) Index/sync into Databricks Vector Search (Delta Sync index). 5) At query time, embed the query and retrieve top-k (Vector Search). 6) Assemble prompt with retrieved context (prompt template). 7) Generate answer via Foundation Model API/Model Serving. 8) Log/trace and evaluate with MLflow. Governance throughout via Unity Catalog.

Q61. MCQ

Which Databricks component provides centralized governance (permissions, lineage) for models, data, and vector indexes used by a deployed RAG app?

- A. Unity Catalog
- B. Photon
- C. Koalas
- D. DBFS root

✓ **Answer: A**

Unity Catalog governs tables, models, functions, and vector indexes with unified permissions and lineage. Photon is an engine; Koalas is a pandas API; DBFS root is legacy storage, not a governance layer.

Q62. MCQ

An app has unpredictable, bursty traffic and you want to avoid paying for idle capacity. Which serving choice fits?

- A. Provisioned Throughput with large reserved capacity
- B. Pay-per-token Foundation Model API (or scale-to-zero serving)
- C. A fixed always-on GPU VM
- D. Batch-only inference

✓ **Answer: B**

Bursty/low-predictability traffic suits pay-per-token (or scale-to-zero endpoints) so you don't pay for idle reserved capacity. Provisioned throughput and always-on VMs waste money when idle; batch-only can't serve interactive requests.

Q63. OPEN

What is the AI Gateway in Databricks and name three governance/ops features it provides.

🔗 **Model answer:**

AI Gateway is a governance/management layer in front of model endpoints. Features include: rate limiting (per user/endpoint), usage tracking via usage tables, request/response logging via inference tables, plus capabilities like permission control, payload guardrails, and routing/fallback. It centralizes cost control, observability, and access policy across LLM endpoints.

Q64. MCQ

Where are an endpoint's request/response payloads automatically captured for monitoring and later evaluation?

- A. Inference Tables
- B. The system prompt
- C. A CSV on the desktop
- D. The embedding index

✓ **Answer: A**

Inference Tables log endpoint requests/responses to Delta for monitoring, debugging, and offline evaluation. The other options don't capture production traffic.

Q65. MCQ

A developer used a Provisioned Throughput endpoint but realizes request volume is too low to justify it. Best change?

- A. Keep it and accept idle cost
- B. Switch to pay-per-token Foundation Model API
- C. Add more provisioned units
- D. Disable monitoring

✓ **Answer: B**

Low volume is the classic case for pay-per-token, avoiding reserved-capacity waste. Keeping idle capacity or adding units increases cost; disabling monitoring is unrelated and unwise.

Q66. OPEN

Why register models to Unity Catalog rather than the legacy Workspace Model Registry for production GenAI apps?

🔗 **Model answer:**

Unity Catalog gives cross-workspace governance, fine-grained permissions, lineage, and a three-level namespace (catalog.schema.model), plus integration with governed data and vector indexes. The legacy workspace registry is workspace-scoped with weaker governance. UC is the recommended path for production.

Q67. MCQ

Which is a valid way to control cost on a Model Serving endpoint?

- A. Enable scale-to-zero and set rate limits via AI Gateway
- B. Always run maximum concurrency
- C. Remove the model signature
- D. Increase temperature

✓ **Answer: A**

Scale-to-zero stops paying for idle, and AI Gateway rate limits cap runaway usage/cost. Max concurrency raises cost; removing signatures and raising temperature don't control cost.

Q68. MCQ

In CI/CD for a GenAI app, what should typically gate promotion from staging to production?

- A. Automated evaluation (quality/faithfulness metrics) plus tests passing
- B. The number of Slack messages
- C. Model file size only
- D. Random approval

✓ **Answer: A**

Promotion should be gated on automated evaluation results and tests (e.g., MLflow evaluate thresholds), ensuring quality before production. The other options aren't quality signals.

Q69. OPEN

Describe how a Delta Sync Vector Search index stays consistent with source data in a deployed app.

🔗 **Model answer:**

The index is bound to a source Delta table (with Change Data Feed). As rows are inserted/updated/deleted, Vector Search incrementally re-embeds and updates the corresponding vectors (continuously or on a schedule), keeping retrieval aligned with current data without manual full re-indexing.

Q70. MCQ

You need to A/B test two prompt versions in production. Which combination supports this cleanly?

- A. Prompt Registry versions/aliases + endpoint traffic splitting + inference logging
- B. Editing the prompt by hand on each request
- C. Two separate companies
- D. Disabling MLflow

✓ **Answer: A**

Registered prompt versions/aliases let you reference variants; traffic splitting routes a fraction to each; inference tables capture outcomes to compare. Manual edits aren't reproducible; disabling MLflow removes the tracking you need.

Q71. MCQ

What does 'scale-to-zero' on a serving endpoint mean?

- A. The endpoint deletes the model
- B. Compute scales down to zero when idle and spins up on the next request (cold start)
- C. It caps tokens at zero
- D. It disables logging

✓ **Answer: B**

Scale-to-zero releases compute during inactivity to save cost, then cold-starts on demand (with some startup latency). It doesn't delete the model, cap tokens, or affect logging.

Q72. OPEN

What is MLflow Tracing and why is it valuable for deployed agent/RAG apps?

🔗 **Model answer:**

MLflow Tracing records the step-by-step execution of a GenAI app — each LLM call, retrieval, tool invocation, inputs/outputs, latency, and tokens — as structured spans. For deployed agents/RAG it gives observability to debug failures, find latency/cost hotspots, inspect retrieved context, and feed evaluation. It's central to MLflow 3's GenAI observability.

Q73. MCQ

A multilingual chatbot must fall back to a backup model if the primary endpoint fails. Which layer best implements routing/fallback and rate limits?

- A. AI Gateway
- B. DBFS
- C. A spreadsheet
- D. The embedding model

✓ **Answer: A**

AI Gateway centralizes routing, fallback, rate limiting, and logging across endpoints. DBFS, spreadsheets, and embedding models don't provide endpoint routing/governance.

Q74. MCQ

Which artifact must accompany a logged model so the serving layer knows input/output structure?

- A. Model signature
- B. A README screenshot
- C. The training GPU serial number
- D. A meme

✓ **Answer: A**

The model signature declares the I/O schema used for validation at serving time. The other items are irrelevant to serving contracts.

Q75. OPEN

Explain the difference between Provisioned Throughput and pay-per-token in terms of cost predictability and capacity.

🔗 **Model answer:**

Provisioned Throughput reserves dedicated capacity for steady, predictable cost and guaranteed throughput/latency at high volume — but you pay even when idle. Pay-per-token charges per request/token with no reserved capacity: cheaper for low/spiky volume and elastic, but cost varies with usage and you don't get dedicated guarantees.

Q76. MCQ

To deploy a custom RAG chain that includes pre/post-processing not covered by a built-in flavor, you should log it as:

- A. A custom MLflow PyFunc model
- B. A plain text file
- C. An image
- D. A Delta table

✓ **Answer: A**

PyFunc wraps arbitrary Python logic into a deployable MLflow model — the standard way to package custom chains. Text/images/Delta tables aren't servable models.

Q77. MCQ

What is the main reason to capture inference tables BEFORE you start formal monitoring/evaluation?

- A. They give you the real production inputs/outputs to evaluate and detect drift on
- B. They reduce model size
- C. They encrypt prompts
- D. They train the model automatically

✓ **Answer: A**

Inference tables provide the ground-truth production traffic needed for monitoring, drift detection, and offline evaluation. They don't shrink models, encrypt, or auto-train.

Q78. OPEN

In a production RAG deployment, where does Unity Catalog touch the pipeline? Name at least three touchpoints.

✍ **Model answer:**

(1) Governs the source/chunk Delta tables (permissions, lineage). (2) Governs the Vector Search index. (3) Registers and governs the served model (catalog.schema.model) and functions/tools. It can also govern inference/usage tables and provide end-to-end lineage from data to model to endpoint.

Q79. MCQ

A deployed endpoint shows rising latency at peak hours. Which is the most appropriate first scaling response?

- A. Increase concurrency/replicas (autoscaling) or move to provisioned throughput
- B. Delete the model
- C. Lower the embedding dimension
- D. Remove guardrails

✓ **Answer: A**

Latency under load is addressed by scaling out (more concurrency/replicas) or reserving throughput. Deleting the model, changing embeddings, or removing guardrails don't solve capacity-driven latency.

Q80. **OPEN**

What is the difference between batch inference and real-time serving for a GenAI app, and give one use case for each?

 Model answer:

Real-time serving exposes a low-latency endpoint for interactive, per-request calls (e.g., a live chatbot). Batch inference processes large datasets offline on a schedule for throughput, not latency (e.g., nightly summarization or classification of millions of documents). Choose real-time for interactivity, batch for high-volume, latency-tolerant jobs — often using `ai_query`/Model Serving for batch scoring.

Section 5 — Governance, Security & Privacy (8%)

Q81. MCQ

Before embedding customer support tickets that contain names and card numbers, what should you do FIRST?

- A. Embed as-is for accuracy
- B. Detect and mask/redact PII
- C. Increase chunk size
- D. Lower temperature

✓ **Answer: B**

PII must be detected and masked/redacted before it enters embeddings, logs, or prompts to meet privacy/compliance. Embedding raw PII creates exposure; chunking and temperature don't address privacy.

Q82. OPEN

How does Unity Catalog enforce least-privilege access for a RAG app's underlying data?

🔗 **Model answer:**

Unity Catalog applies fine-grained, role-based permissions (GRANT/REVOKE) at catalog/schema/table/column/row levels, plus dynamic views and row/column masking. A retrieval service can be limited to only the rows/columns a given principal may see, and lineage shows who accessed what — enforcing least privilege end to end.

Q83. MCQ

A user without rights to HR data queries the chatbot and gets HR answers. Which control most directly prevents this?

- A. Metadata/ACL filtering and Unity Catalog permissions on the retrieved source
- B. A larger LLM
- C. Higher k
- D. More few-shot examples

✓ **Answer: A**

Access should be enforced at the data/retrieval layer via UC permissions and metadata/ACL filters so unauthorized rows are never retrieved. Model size, k, and examples don't enforce authorization.

Q84. MCQ

You plan to use an open model commercially. What governance fact must you verify?

- A. Its color scheme
- B. Its license terms permit commercial use
- C. Its GitHub star count
- D. The author's location

✓ **Answer: B**

Licensing determines whether commercial use, redistribution, or fine-tuning is allowed — a core governance check. Stars, colors, and author location are irrelevant to legal use.

Q85. OPEN

What is the governance risk of logging full prompts/responses in inference tables, and how do you mitigate it?

🔗 **Model answer:**

Logged payloads can contain PII or secrets, creating privacy/compliance exposure. Mitigate by masking/redacting PII before logging, restricting access to the inference tables via Unity Catalog permissions, applying retention policies, and using column masking — so observability doesn't violate privacy.

Q86. MCQ

Which is the BEST practice to prevent prompt-injection from user-supplied content in a deployed app?

- A. Trust all user input
- B. Separate/delimit untrusted input, validate it, and apply input/output guardrails
- C. Raise temperature
- D. Remove the system prompt

✓ **Answer: B**

Delimiting untrusted input, validating it, and adding guardrails reduces injection risk. Trusting input, raising temperature, or removing the system prompt all increase vulnerability.

Q87. MCQ

A regulation requires you to show which source documents informed a generated answer. Which capability supports this?

- A. Citations/traceability from retrieved chunks plus Unity Catalog lineage
- B. Higher temperature
- C. Disabling logging
- D. Bigger embeddings

✓ **Answer: A**

Returning citations to retrieved sources and relying on UC lineage gives auditable traceability. Temperature, disabling logs, and embedding size don't provide provenance.

Q88. OPEN

Differentiate column masking and row filtering in Unity Catalog and give a GenAI use case for each.

🔗 **Model answer:**

Column masking hides/transforms sensitive column values (e.g., show only last 4 digits of an SSN) — useful so a RAG source view never exposes raw PII. Row filtering restricts which rows a principal can see (e.g., a sales rep only sees their region's tickets) — useful so retrieval returns only authorized records. Both enforce data-layer governance independent of the model.

Section 6 — Evaluation & Monitoring (12%)

Q89. MCQ

Which metric measures whether a RAG answer is supported by the retrieved context (no hallucination)?

- A. Faithfulness / groundedness
- B. BLEU
- C. QPS
- D. Perplexity

✓ **Answer: A**

Faithfulness (groundedness) checks that the generated answer is entailed by the retrieved context. BLEU is n-gram overlap for translation/summarization; QPS is throughput; perplexity is a language-model fit metric.

Q90. OPEN

Distinguish Context Precision, Context Recall, and Answer Relevancy in RAG evaluation.

✍ **Model answer:**

Context Precision: of the retrieved chunks, how many are actually relevant (retriever signal-to-noise).

Context Recall: of the relevant information that exists, how much was retrieved (did we miss anything).

Answer Relevancy: does the generated answer actually address the user's question. Precision/recall judge retrieval; relevancy judges the generation against the query.

Q91. MCQ

For a classification task you care about catching as many true positives as possible even at some false-positive cost. Which metric do you prioritize?

- A. Recall
- B. Precision
- C. BLEU
- D. ROUGE-L

✓ **Answer: A**

Recall measures coverage of actual positives (minimizing false negatives) — the priority when missing positives is costly. Precision penalizes false positives; BLEU/ROUGE are generation metrics.

Q92. MCQ

You want to evaluate a RAG chain's quality programmatically on Databricks with built-in LLM judges. Which do you use?

- A. `mlflow.evaluate()` with GenAI/Agent metrics
- B. print statements
- C. A bigger embedding model
- D. Photon

✓ **Answer: A**

`mlflow.evaluate()` runs evaluation (incl. LLM-as-judge metrics like faithfulness, relevance, correctness) over your RAG/agent outputs. Print statements aren't evaluation; embeddings and Photon don't evaluate quality.

Q93. OPEN

What is LLM-as-a-Judge, and name two pitfalls the exam expects you to know.

🔗 **Model answer:**

LLM-as-a-Judge uses an LLM to score outputs (relevance, faithfulness, correctness) against a rubric/reference. Pitfalls: (1) bias — position/verbosity/self-preference bias and inconsistency; (2) cost/latency and the need for a strong judge model plus calibrated prompts and, ideally, human spot-checks. Mitigate with clear rubrics, reference answers, and validation against human labels.

Q94. MCQ

Which production metric tells you how fast your endpoint responds per request?

- A. Latency
- B. Faithfulness
- C. ROUGE-1
- D. Context recall

✓ **Answer: A**

Latency is the per-request response time, a core operational monitoring metric. Faithfulness/ROUGE/context recall are quality metrics, not speed.

Q95. MCQ

Which Lakehouse Monitoring monitor type is designed for ML/GenAI endpoints logging requests over time with model-quality metrics?

- A. InferenceLog monitor
- B. Snapshot monitor
- C. TimeSeries monitor
- D. Schema monitor

✓ **Answer: A**

The InferenceLog monitor is built for inference tables — tracking quality and drift on model requests/responses over time. Snapshot monitors a table's current state; TimeSeries tracks generic time-stamped metrics; 'Schema monitor' isn't a Lakehouse Monitoring type.

Q96. OPEN

What is a custom scorer in Databricks evaluation, and when would you create one?

🔗 **Model answer:**

A custom scorer is a user-defined evaluation function (heuristic or LLM-judge based) that returns a metric for each example, registered into the evaluation harness. Create one when built-in metrics don't capture domain-specific quality — e.g., 'does the answer cite a valid policy number,' 'is the JSON schema-valid,' or a business-specific correctness rule.

Q97. MCQ

BLEU and ROUGE differ mainly in that:

- A. BLEU emphasizes precision of n-grams (often translation); ROUGE emphasizes recall (often summarization)
- B. They are identical
- C. BLEU measures latency; ROUGE measures cost
- D. Both measure retrieval recall

✓ **Answer: A**

BLEU is precision-oriented n-gram overlap common in MT; ROUGE is recall-oriented, common in summarization (ROUGE-N, ROUGE-L). They are not identical and don't measure latency, cost, or retrieval.

Q98. MCQ

How is Subject-Matter-Expert (SME) feedback BEST incorporated into a Databricks GenAI evaluation loop?

- A. Ignore it once deployed
- B. Capture SME labels/feedback (via review apps/feedback tables) and use them to calibrate judges and build eval datasets
- C. Only use BLEU
- D. Increase temperature based on mood

✓ **Answer: B**

SME feedback is collected (e.g., through Agent Evaluation review apps/feedback tables) and used to build/curate golden evaluation datasets and calibrate LLM judges. Ignoring it or relying on a single automatic metric undercuts quality.

Q99. OPEN

Why monitor cost and token usage alongside quality metrics in production GenAI, and which Databricks features expose this?

🔗 **Model answer:**

GenAI cost scales with tokens and traffic, so cost/usage must be tracked to prevent budget blowouts and to balance quality vs. spend. Databricks exposes this via AI Gateway usage tables, inference tables (for token counts/latency), and Lakehouse Monitoring dashboards (QPS, latency, cost, error rate, feedback).

Q100 MCQ

Mosaic AI Agent Evaluation is distinct from the Agent Framework because it primarily:

- A. Builds the agent
- B. Provides evaluation (quality metrics, LLM judges, review apps) for agent/RAG outputs
- C. Serves the endpoint
- D. Stores embeddings

✓ **Answer: B**

Agent Evaluation focuses on measuring agent/RAG quality with metrics, LLM judges, and human review apps; the Agent Framework is for building/deploying agents. They are complementary but distinct. It does not itself serve endpoints or store embeddings.